

ASD 2026 – Project Group Registration Form

Submission Deadline: End of Week 4

Submit the completed form to your tutor after all team members have enrolled in the appropriate Canvas Group.

Section 1 – Team Information

Canvas Group Number	[enter your Canvas Group number]
Team Leader	[enter team leader name]
Team Leader Email	[enter team leader UTS email]
Chosen Project Topic	Prediction Market Intelligence Hub An Agentic AI microservices application for prediction-market analysis (Finance & Analytics domain). Users track, research, and analyse prediction markets (Polymarket / Kalshi style) through a single unified, AI-assisted web portal. All market data is seeded mock/historical data so local, CI, and cloud demos do not depend on a live third-party feed.
Project GitHub Repository URL	https://github.com/<your-org-or-username>/prediction-market-hub (create, then paste the final URL here)
Selected Cloud (AWS / Azure)	Amazon Web Services (AWS) — Amazon ECS with AWS Fargate
Approved Open-Source LLM(s)	To be finalised — approved options: Llama 3.1 8B, Qwen2.5 7B, or DeepSeek (via Ollama). Repo currently defaults to Llama 3.1 8B.

Section 2 – Team Members

No.	Student Name	Student ID	UTS Email
1	[full name]	[student ID]	[UTS email]
2	[full name]	[student ID]	[UTS email]
3	[full name]	[student ID]	[UTS email]
4	Erfanur Rahman (Erfy)	[student ID]	[UTS email]
5	N/A — this team has four (4) members (approved four-person project structure).		

Section 3 – Individual Feature Selection

Student 1 Name	[assign a team member]
Feature Name	Market Watchlist & Discovery
Feature Purpose	Lets users browse and search prediction markets by category (politics, sports, crypto, economics) and maintain a personal watchlist of tracked markets, with an AI-Mode summary of what is trending.
Frontend Functions	• Market browse/list page with category filter and keyword search • Market detail view (title, category, current probability, volume, close date) • Add-to-watchlist and remove-from-watchlist controls • Watchlist page showing all tracked markets with current (seeded) prices • "Ask AI: what markets are trending?" panel (AI-Mode)
Backend/API Functions	• GET /markets — list, filter by category, keyword search

	(Read) • GET /markets/{id} — retrieve a single market (Read) • POST /watchlist — add a market to the watchlist (Create) • DELETE /watchlist/{id} — remove from watchlist (Delete) • PUT /watchlist/{id} — update a watchlist entry, e.g. note/priority (Update) • AI: POST /ai/trending — Ollama summarises trending / high-movement markets from seeded data (Plan → Act → Observe → Adapt)
Database Tables	• markets — market metadata: title, category, current probability, volume, close date (≥10 records) • watchlist — user's tracked market references + optional note/priority (≥10 records) • categories — lookup of market categories (≥10 records)

Student 2 Name	<i>[assign a team member]</i>
Feature Name	Portfolio & Position Tracker
Feature Purpose	Lets users log hypothetical positions (market, side, entry price, size) and track profit/loss and exposure by category over time, with an AI-generated portfolio summary.
Frontend Functions	• New-position form (market, side, entry price, size) • Positions table (open and closed) with edit / close / delete actions • P&L and exposure-by-category dashboard with simple charts • "Ask AI: summarise my portfolio" panel (AI-Mode)
Backend/API Functions	• POST /positions — log a new position (Create) • GET /positions — list open and closed positions (Read) • PUT /positions/{id} — update / close / resize a position (Update) • DELETE /positions/{id} — remove a position (Delete) • GET /portfolio/summary — aggregate P&L and exposure by category • AI: POST /ai/portfolio-summary — Ollama produces a natural-language performance overview
Database Tables	• positions — market ref, side, entry price, size, status, opened/closed dates (≥10 records) • portfolio_snapshots — periodic P&L / exposure snapshots for trend charts (≥10 records) • position_categories — category exposure lookup / mapping (≥10 records)

Student 3 Name	<i>[assign a team member]</i>
Feature Name	Market Research Notes & News Feed
Feature Purpose	Lets users attach research notes and tagged evidence to a specific market and browse a feed of news snippets linked to that market, with an AI recap of a market's notes.
Frontend Functions	• Note editor to create / edit / delete notes attached to a market • Per-market notes list with tag display and filtering • Tag input for classifying evidence • News feed panel (seeded snippets) filtered by market • "Ask AI: summarise my notes on this market" panel (AI-

	Mode)
Backend/API Functions	● POST /notes — create a research note on a market (Create) ● GET /notes?market_id= — list notes for a market (Read) ● PUT /notes/{id} — edit a note (Update) ● DELETE /notes/{id} — delete a note (Delete) ● GET /news?market_id= — retrieve the news feed for a market ● AI: POST /ai/notes-summary — Ollama recaps a market's notes
Database Tables	● notes — market ref, body, tags, created/updated timestamps (≥10 records) ● news_items — market ref, headline, snippet, source, published date (≥10 records) ● tags — lookup of note/evidence tags (≥10 records)

Student 4 Name	Erfanur Rahman (Erfy)
Feature Name	AI Market Analyst Assistant
Feature Purpose	Provides an AI chat interface that explains price movements and summarises notes/news for a market. Each analysis run is saved as an editable/deletable Analysis Report (the feature's core CRUD entity), and the assistant flags potentially mispriced or highly volatile markets.
Frontend Functions	● AI chat interface scoped to a selected market ● Run-analysis button that saves each run as an Analysis Report ● Saved-reports list with view / edit / delete ● "Flagged markets" panel (mispriced / high-volatility)
Backend/API Functions	● POST /analysis — run analysis and save it as a report (Create) ● GET /analysis — list / retrieve saved analysis reports (Read) ● PUT /analysis/{id} — edit a saved report (Update) ● DELETE /analysis/{id} — delete a report (Delete) ● AI: POST /ai/analyse — Ollama Plan → Act → Observe → Adapt loop over seeded market / notes / news ● GET /analysis/flags — volatility / mispricing heuristics with AI commentary
Database Tables	● analysis_reports — market ref, prompt, AI output, verdict, created/updated (≥10 records) ● flags — market ref, flag type, score, rationale (≥10 records) ● ai_runs — log of Plan / Act / Observe / Adapt steps per run (≥10 records)

Student 5 Name	<i>N/A — four-member team (approved four-person structure)</i>
Feature Name	<i>Not applicable.</i>

Individual Responsibilities

Each student is individually responsible for:

- Developing and maintaining one frontend microservice.
- Developing and maintaining one backend/API microservice.
- Developing and maintaining one database microservice.
- Implementing CRUD operations through the frontend and backend microservices.
- Populating each database table with a minimum of ten (10) records.

- Integrating the frontend with the backend API to interact with the Agentic AI models.
- Integrating their assigned microservices into the team's shared GitHub project.
- Implementing and maintaining their own CI/CD workflow.
- Maintaining GitHub commit history and GitHub Actions CI/CD workflow execution.
- Demonstrating the Plan → Act → Observe → Adapt Agentic AI workflow.
- Maintaining pre-testing and post-testing evidence for the assigned microservices.

Team Responsibilities

Each team is responsible for:

- Developing one integrated Agentic AI software application.
- Integrating all individual frontend, backend, and database microservices into a single working AI application.
- Ensuring every frontend and backend microservice demonstrates successful interaction with the approved LLM(s) during each showcase.
- Maintaining a shared GitHub repository with a common project structure.
- Maintaining a shared Docker Compose configuration for integrated execution of all microservices and AI services.
- Providing a unified home page (index.html) that links all individual frontend features.
- Applying a consistent CSS theme and user interface across the application.
- Integrating and testing all team microservices before each release.
- Resolving integration issues before each showcase.
- Maintaining project documentation throughout all releases.
- Demonstrating the integrated application from a single team member's computer during each showcase.
- Ensuring each team member demonstrates their assigned feature within the integrated application.
- Submitting one technical report (PDF) per release on Canvas (one per group).

Table 4 – Team Declaration and Signatures

We, the undersigned, confirm that we have formed this project team, enrolled in the appropriate Canvas Group, selected the proposed project topic, registered our individual project features, and established the GitHub repository.

Student Name	Signature	Date
[name]		
[name]		
[name]		
[name]		

Tutor Use Only

Date Received	
Tutor Name	
Approved	
Comments	